

ALEXA VOICE ASSISTANT



Prepared by: Divya Upadhyay

02/10/2026

GitHub Link: [\[GitHub\]](#)

Outline

- Executive
- Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary (Methodologies)

- **Speech Recognition:** Captured voice commands using sounddevice and converted speech into text using **Whisper**.
- **Text Preprocessing & Feature Extraction:** Used **TF-IDF** to convert transcribed text into numerical feature vectors.
- **Intent Classification:** Applied **MLPClassifier** to identify the user's intent from the TF-IDF features.
- **Action Mapping:** Connected predicted intents to specific actions such as opening websites, playing music, showing news, and answering general queries.
- **Response Generation:** Generated response text based on the detected intent.
- **Text-to-Speech:** Used **pyttsx3** to convert the response back into spoken audio.
- **Application Development:** Integrated the complete pipeline into an interactive **Streamlit** application.
- **Methodology Flow**
- **Voice Input → Speech-to-Text → TF-IDF → MLP Classification → Intent → Action → Response → Text-to-Speech**
- **GitHub Link:** [\[GitHub\]](#)

Executive Summary (Results)

- Successfully developed a **working Mini Alexa Voice Assistant** using Python and Streamlit.
- Voice commands are converted into text using **Whisper Speech Recognition**.
- **TF-IDF** converts user input into numerical features for machine learning.
- **MLPClassifier** predicts the user's intent from the voice command.
- The system successfully performs implemented actions such as:
 - Opening YouTube/Google
 - Playing music
 - Opening Google News
 - Providing date and time
 - Generating jokes
 - Performing Google searches
- **pyttsx3** converts the generated response into speech.
- The complete system demonstrates an **end-to-end AI pipeline: Voice → Text → Intent → Action → Voice**.
- The project also includes a **Power BI dashboard** for exploring the intent dataset.
- **Key Result:** A functional prototype that connects **Speech Recognition, NLP, Machine Learning, and Automation** into one interactive application.
- **GitHub Link:** [\[GitHub\]](#)

How Alexa works?

Step 1.: User speak 



Step 2.: Speech to Text ( Whisper



Step 3.: - Train Alexa using data



Step 4.: - Understand User Intent using Deep Learning



Step 5.: - define action engine for Alexa



Step 6.: - Text to speech

Introduction

- The **Alexa Voice Assistant** is a mini AI-powered voice assistant developed using **Python**. The project demonstrates how multiple Data Science and AI technologies can work together in a real-world application.
- The system captures a user's voice command, converts speech into text using **Whisper**, extracts meaningful text features using **TF-IDF**, and predicts the user's intent using an **MLPClassifier**. Based on the predicted intent, the system performs a relevant action and provides a spoken response using **pyttsx3**.
- **Key Technologies**
- **Speech Recognition → NLP → Machine Learning → Automation → Text-to-Speech**
- The project was developed as an end-to-end application using **Streamlit**, providing an interactive interface for users to communicate with the voice assistant.
- **GitHub Link:** [\[GitHub\]](#)

Section 1

Methodology

GitHub Link: [\[GitHub\]](#)

The image shows a person's desk setup. A computer monitor displays a calendar for February 26th. A person is holding a white mug. The Alexa+ logo, featuring the word 'alexa' in a sans-serif font and a '+' sign, is overlaid in the center. Below the text is a white curved arrow pointing from left to right, resembling the Amazon smile logo. The background is a soft-focus view of a desk with a plant and a person's arm in a grey shirt.

alexa+

Audio Recording

- The system captures the user's voice through the **microphone** using the sounddevice library.
- Audio is recorded for a configurable duration at a **48,000 Hz sampling rate**.
- The recording uses **one audio channel (mono)** for speech input.
- The recorded audio is saved as a **WAV file** using scipy.io.wavfile.
- The saved audio file is then passed to the **Whisper model** for speech-to-text conversion.
- Proper microphone selection is important to ensure the WAV file contains clear voice input.

- **Flow:**

Microphone → sounddevice → Audio Recording → WAV File → Whisper

- **GitHub Link:** [\[GitHub\]](#)

3:Record Audio From Microphone----

```
import sounddevice as sd # help to record audio
from scipy.io.wavfile import write # help to store audio
```

`record_audio()` : It will Record audio from microphone for 5 seconds, at speech-quality sample rate, using one audio channel.

```
def record_audio(filename = 'input2.wav',duration = 5,fs = 48000):
    print(" Listening")

    recording = sd.rec(int(duration*fs),samplerate=fs,channels=1)
    sd.wait()
    write(filename,fs,recording)
    print("Audio recording")
```

```
record_audio(fs=48000)
```

```
''' Listening
Audio recording
```

```
'''
```

```
If a .wav file has no audio, it usually means Python selected the wrong microphone & as a result, the recorded file contains no voice.
```

```
'''
```

```
''' '\n\nIf a .wav file has no audio, it usually means Python selected the wrong microphone & as a result, the recorded file contains no voice.\n\n'
```

FFmpeg Setup & Audio Processing

- **FFmpeg** was installed to provide the required audio-processing support for **Whisper**.
- The FFmpeg package was downloaded and extracted into the project environment.
- The required FFmpeg directory was added to the system **PATH** using Python.
- The installation was verified by checking the FFmpeg executable and version.
- This setup enables Whisper to properly process the recorded **WAV audio file** for speech recognition.

- **Flow:**

**Download FFmpeg → Extract → Add to PATH
→ Verify Installation → Process Audio with
Whisper**

- **GitHub Link:** [\[GitHub\]](#)

```
Lets install ffmpeg!!

import os
import subprocess
import urllib

url = "https://www.gyan.dev/ffmpeg/builds/ffmpeg-release-essentials.zip"
#create directory where we can store our ffmpeg
os.makedirs("ffmpeg", exist_ok=True)

urllib.request.urlretrieve(url, r"ffmpeg\ffmpeg.zip") # Open the directory ffmpeg as if it were a file and
('ffmpeg/ffmpeg.zip', <http.client.HTTPMessage at 0x2c188032060>)

os.environ['PATH']

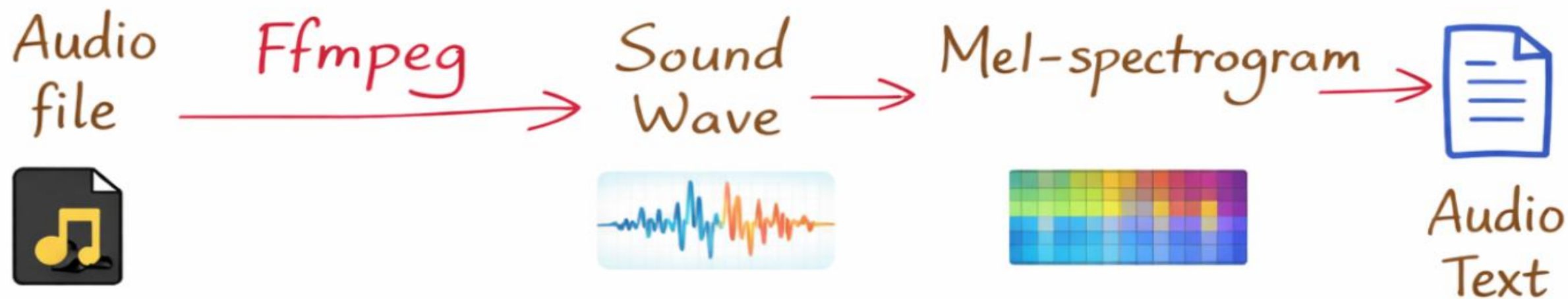
'c:\Users\divya\anaconda3\envs\ml_env;C:\oraclexe\app\oracle\product\10.2.0\server\bin;C:\Prog

os.pathsep

';'

print(os.listdir(r"ffmpeg\ffmpeg-9.0.2-essentials build"))
```

Speech to Text



Speech-to-Text using Whisper

- The recorded **WAV audio** is provided as input to the **OpenAI Whisper** model.
- The **Whisper Base** model processes the audio and converts spoken words into text.
- The transcribed text is extracted from the model's result and converted to **lowercase** for further processing.
- The resulting text becomes the input for the next stage: **Intent Classification**.
- Example: the recorded voice command is converted into text such as “**Please open your tomb.**”
- On CPU, Whisper automatically uses **FP32 instead of FP16**, as shown by the warning in the output.
- **Flow:**
WAV Audio → Whisper Base → Speech Recognition → Transcribed Text → Intent Classification
- **GitHub Link:** [\[GitHub\]](#)

5 Speech-to-text using OpenAI Whisper----

```
import whisper
```

```
model = whisper.load_model("base")
```

```
result = model.transcribe("input2.wav")
```

```
c:\Users\divya\anaconda3\envs\ml_env\lib\site-packages\whisper\transcribe.py:132: UserWarning: FP16 is not supported on CPU; using FP32 instead
warnings.warn("FP16 is not supported on CPU; using FP32 instead")
```

```
text = result['text']
text
```

```
' Please open your tomb.'
```

[Generate](#)[+ Code](#)[+ Markdown](#)

```
def speech_to_text(audio_path):
    result = model.transcribe(audio_path)
    text = result['text']
    print("you said:", text)
    return text.lower()
```

```
speech_to_text('input2.wav')
```

```
c:\Users\divya\anaconda3\envs\ml_env\lib\site-packages\whisper\transcribe.py:132: UserWarning: FP16 is not supported on CPU; using FP32 instead
warnings.warn("FP16 is not supported on CPU; using FP32 instead")
```

Feature Extraction using TF-IDF

- The transcribed user prompts are converted into **numerical features** using **TF-IDF** (Term Frequency–Inverse Document Frequency).
- `TfidfVectorizer` learns the important words from the dataset and transforms the text into a **numerical feature matrix**.
- The dataset contains **4,785 prompts**, which are represented using **192 TF-IDF features**.
- The resulting matrix has a shape of **(4785, 192)**.
- Each row represents one user prompt, while each column represents a TF-IDF feature.
- This numerical representation is then provided to the **MLPClassifier** for intent prediction.

- **Flow:**

Text Prompts → `TfidfVectorizer` → Numerical Feature Matrix → `MLPClassifier`

- **GitHub Link:** [\[GitHub\]](#)

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer()
```

```
X = vectorizer.fit_transform(alexa_df['prompt'])
```

```
X.shape
```

```
(4785, 192)
```

```
X.toarray()
```

```
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]], shape=(4785, 192))
```

```
X.toarray()[0]
```

Intent Classification using MLPClassifier

- The project uses **MLPClassifier (Multi-Layer Perceptron)** as a simple neural-network model for intent classification.
- It learns patterns from the **TF-IDF feature vectors** to predict the user's intent.
- The model uses **two hidden layers with 50 and 25 neurons**.
- `max_iter=500` allows the model up to **500 training iterations**.
- The model is trained using the TF-IDF feature matrix **X** and the corresponding **intent labels**.
- After training, the model can classify new user commands into their learned intent categories.

- **Flow:**

TF-IDF Features → MLP Neural Network → Learned Patterns → Predicted Intent

- **GitHub Link:** [\[GitHub\]](#)

7 Train a simple deep learning intent model ----

```
[54] from sklearn.neural_network import MLPClassifier

...
MLP = Multi-Layer Perceptron
In simple words:
👉 It's a basic neural network that learns patterns in data to make decisions.
...

[55] ...
'\nMLP = Multi-Layer Perceptron\nIn simple words:\n👉 It's a basic neural network that learns patterns in data to make decisions.\n\n'

[56] intent_model = MLPClassifier(hidden_layer_sizes=(50,25),max_iter=500)

[57] intent_model.fit(X,alex_df['intent'])

...
MLPClassifier ⓘ ⓘ
  Parameters
  Fitted attributes
```

Intent

Shopping



Order from
Amazon

amazon

Alarm



Set alarm
in phones



Weather



21°C



Music



Play song on
YouTube

YouTube

Action Engine

- be performed.
- For play_music, it opens **YouTube** and searches for music.
- For open_website, it identifies the requested website and can open **YouTube, Google, or GitHub and for more work**
- For jokes_fun, it selects a joke from a predefined list using random.choice().
- For news, it opens **Google News** to provide current headlines.
- The system also returns a **text response** describing the action performed.
- This connects the **ML prediction** with a real-world application action.
- Flow
- Predicted Intent → Action Engine → Perform Action → Generate Response
- **GitHub Link:** [\[GitHub\]](#)

```
shopping_list = []

def perform_action(intent, prompt = ""):

    #play music
    if intent == 'play_music':
        webbrowser.open("https://www.youtube.com/results?search_query=music")
        return "Playing music on Youtube"

    # for opening website
    elif intent == 'open_website':
        prompt_lower = prompt.lower()

        #remove punctuation for safer matching
        cleaned = re.sub(r'^a-zA-Z0-9\s', prompt_lower)

        if "youtube" in cleaned or "you tube" in cleaned:
            webbrowser.open("https://www.youtube.com")
            return "Opening Youtube"
        elif "google" in prompt_lower:
            webbrowser.open("https://www.google.com")
            return "Opening Google"
        elif "github" in prompt_lower:
            webbrowser.open("https://www.github.com")
            return "Opening Github"
        else:
            return "which website would you like to open"

    #jokes
    elif intent == 'jokes_fun':
        jokes = [
            "Why did the computer get cold? Because it forgot to close Windows!",
            "Why do programmers prefer dark mode? Because light attracts bugs!",
            "I told my AI assistant a joke... it said, processing humor module."
        ]
        return random.choice(jokes)

    #news
    elif intent == "news":
        webbrowser.open("https://news.google.com")
```

Text-to-Speech & Complete Assistant Pipeline

- **pyttsx3** is used to convert the generated text response into spoken audio.
- The **SAPI5** speech engine is initialized for voice output.
- The available system voices are retrieved, and a voice is selected.
- Speech **rate** is set to 165 and **volume** to 1.0.
- The `speak()` function converts the response text into speech using `engine.say()` and `engine.runAndWait()`.
- The `run_assistant()` function connects all major components into one complete pipeline:
Record Audio → Speech-to-Text → Intent Prediction → Action Engine → Voice Response
- This creates the final **end-to-end voice assistant workflow**.
- **Complete Flow**
- 🎤 Voice Input → Whisper → Text → MLPClassifier → Intent → Action → Response → pyttsx3 → 🔊 Voice Output
- **GitHub Link:** [\[GitHub\]](#)

```
import pyttsx3

engine = pyttsx3.init('sapi5') ## initialize Text-to-Speech engine & uses built-in

voices = engine.getProperty('voices') ## Fetches all voices installed on the OS

engine.setProperty('voice', voices[1].id) ## 1 -> Load usually female voice
engine.setProperty('rate', 165) ## 1 -> Load usually female voice
engine.setProperty('volume', 1.0)

def speak(text):
    print("Alexa:", text)
    engine.say(text)
    engine.runAndWait()

def run_assistant():
    record_audio('input2.wav')
    text = speech_to_text('input2.wav')
    intent = predict_intent(text)
    response = perform_action(intent, text)
    speak(response)

run_assistant()
```



alexa

Thank you!